



4



14



1680 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Relational Algebra part 2

د. روان قرعوني



RB Informatics; 28/5/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات

تعرفنا في المحاضرة السابقة على مفهوم الجبر العلاقي الذي يمثل الأساس النظري للغات الاستعلام

مثل SQL و تعرفنا على العمليات الأساسية ضمنه :

الاختيار - الاسقاط - التقاطع/الاجتماع - الفرق - الضرب الديكارتي - إعادة التسمية

في هذه المحاضرة :

سنتعرف ثلاث على عمليات مهمة جداً نحتاجها عند التعامل مع علاقات أكثر تعقيداً، و هي :

- الانضمام (Join) ∞
- القسمة (Division) $\div$
- الإسناد (Assignment) $\leftarrow$

الانضمام (Join) :

الرمز : ∞

- هي عملية تستخدم لدمج صفوف من جدولين مختلفين في علاقة واحدة، و ذلك من خلال ربط البيانات ذات الصلة بين الجدولين.
- تعتبر هذه العملية أداة أساسية لاسترجاع معلومات متكاملة موزعة على أكثر من جدول.
- هي تطوير لفكرة الضرب الديكارتي، لكنها أكثر دقة لأنها لا تحتفظ بكل التركيبات، بل فقط المطابقات.

الصيغة العامة :

$$S \infty R$$

فائدة أربيسيزية !

عند استخدام صيغة الانضمام بدون وجود شرط، مثل $S \infty R$ ، فإننا نشير إلى نوع محدد من أنواع الانضمام يعرف بـ **الانضمام الطبيعي (Natural Join)**. حيث تتم مطابقة الصفوف تلقائياً بناءً على الأعمدة ذات الأسماء والقيم المتطابقة في الجدولين. معلومة إضافية: يوجد عدة أنواع من الانضمام في الجبر العلاقي، مثل: الانضمام الشرطي (Theta Join). الانضمام المتساوي (Equi Join)

تُكمل .. بفرض R و S جدولان :

العملية تربط بين الأعمدة المتشابهة في R , S .

بفرض الجدول R

E	B	D
....		
....		

∞

بفرض الجدول S

A	B	C	D
...			...
.....			

⌞ عملية الانضمام بينهما تكافئ :

$$S \infty R = \sigma R.B = S.B \wedge R.D = S.D (R \times S)$$

✓ مثل الضرب الديكارتي، لكن يُبقي فقط الصفوف التي تحقق شرط الربط.

✓ تُستخدم كثيرًا لدمج المعلومات من أكثر من جدول عند الاستعلام.

(القسمة العلاقية) (Relational Division) :

الرمز : ÷

■ هي عملية تُستخدم عندما نريد استخراج العناصر التي ترتبط بجميع عناصر مجموعة أخرى.

الصيغة العامة :

$$R \div S$$

بفرض الجدول R

a	1
b	2
a	2
c	4

÷

بفرض الجدول S

1
2

⌞ ناتج القسمة $R \div S$ هو العناصر من R المرتبطة بجميع القيم في S .

✓ تُستخدم للإجابة على الأسئلة من نوع "لكل" (For all).

✓ هذه الأسئلة لا يمكن تحقيقها باستخدام التقاطع أو الاختيار فقط.

مثل : "من هم الطلاب الذين حضروا جميع الدورات؟"

الإسناد (Assignment) :

الرمز : ←

- هو عملية تخزين نتيجة علاقة (استعلام) في اسم جديد مؤقت، يشبه تعريف متغير مؤقت في البرمجة، لتسهيل التعبيرات المعقدة وإعادة استخدامها دون تكرار .
- يساعد على تنظيم العمليات المركبة وتبسيط الكتابة.

الصيغة العامة :

$$Temp \leftarrow \sigma \text{ condition}(R)$$

ثم نستخدم Temp في عمليات لاحقة دون الحاجة إلى إعادة كتابة التعبير.

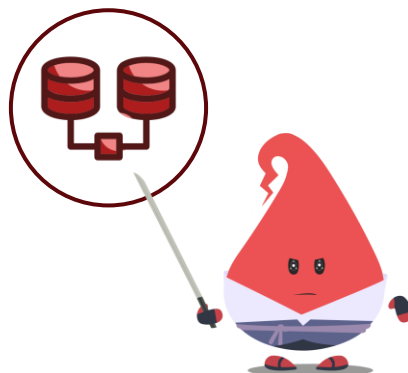
- ✓ يجعل التعبيرات الطويلة أسهل في القراءة.
- ✓ يشبه تعريف "متغير مؤقت" لحفظ نتيجة بسيطة.

الفرق بين إعادة التسمية والإسناد في الجبر العلاقتي :

إعادة التسمية (Renaming)	الإسناد (Assignment)	VS
ρ (rho)	←	الرمز المستخدم
تغيير اسم الجدول و/أو أعمدته	تخزين نتيجة استعلام باسم مؤقت (متغير)	الوظيفة الأساسية
لا، تُغير فقط الاسم	نعم، تُخزن نتيجة استعلام في علاقة جديدة	هل تنشئ علاقة جديدة؟
تنظيم وتوضيح الأسماء في الاستعلامات المركبة	تبسيط العبارات الطويلة وتكرار الاستخدام	الغرض
تغيير اسم الجدول أو الحقل فقط	تعريف متغير يحتوي على نتيجة	تشبه في البرمجة؟

متى نستخدم كل واحدة ؟

- استخدم إعادة التسمية (ρ) عندما تريد تغيير اسم الجدول أو الأعمدة داخل استعلام مركب لتجنب التداخل أو الغموض، خصوصاً في Self Join.
- استخدم الإسناد (←) عندما تريد إعادة استخدام نتيجة معينة لاحقاً أو لتقسيم الحل إلى خطوات منظمة.



تمرين شامل

ليكن لدينا نظام مصرف يحوي الكيانات التالية والواصفات الخاصة بكل واحدة منها و نريد الاستعلام عنها باستخدام الجبر العلاقتي:

- الفرع له اسم و مدينة (Branch (branch-name,branch-city)
- الزبون له اسم و عنوان (Customer (customer-name,customer-street)
- الحساب له رقم و فرع و رصيد (Account (account-number,branch-name,balance)
- القرض له رقم و فرع و المبلغ (Loan (loan-number,branch-name,amount)
- حساب المودع له اسم الزبون و رقم الحساب (Depositor (customer-name,account-number)
- حساب المقرض له اسم الزبون و رقم القرض (Borrower (customer-name,loan-number)

1. الاستعلام عن القروض التي قيمتها أكبر من 1000:

نريد جميع الصفوف في جدول القروض التي تحوي مبلغ اكبر من 1000

✂️ عملية اختيار selection

$\sigma_{\text{amount} > 1000}(\text{loan})$

2. أعط رقم القرض لكل قرض قيمته أكبر من 1000:

نتيجة الطلب الأول تعطي جميع الصفوف في جدول القروض المحققة للشرط (المبلغ اكبر من 1000) متضمنة جميع الأعمدة في هذا الجدول (رقم القرض و الفرع و المبلغ) و في هذا الطلب نحتاج الى ارقام القروض حصرا == عمود محدد من النتيجة السابقة

✂️ عملية اسقاط projection

$\pi_{\text{loan-number}}(\sigma_{\text{amount} > 1000}(\text{Loan}))$



Rockets aren't judged by their size 🤖🚀
but by their precision in aiming at the target

3. أسماء كل الزبائن التي لديها قرض أو حساب:

أولاً : الزبائن التي لديها حسابات:

للوصول الى أسماء الزبائن التي تملك حسابات ننظر أولاً الى جدول الحسابات فنجد أنه لا يحوي اسم الزبون بشكل مباشر و لكنه يحوي account-number الذي من خلاله نستطيع جلب اسم الزبون عن طريق جدول المودعين (Depositor)

بشكل آخر : نحتاج إلى الربط بين جدول الحسابات وجدول العملاء عبر Depositor لجلب أسماء العملاء.

عملية ضرب ديكارتي بين Depositor و Account

Depositor × Account

شكل الجدول الناتج عن هذه العملية :

Account			Depositor	
Account-number	Branch-name	balance	Customer-name	Account-number
.....				

و لكن ذكرنا سابقاً أن الضرب الديكارتي لا يستخدم بمفرده و إنما نحتاج الى إتباعه بعملية اختيار او اسقاط للحصول على النتائج المطلوبة . و في حالتنا هنا نحتاج الى اختيار الصفوف التي فيها :

Account.account-number = Depositor.account-number

عملية اختيار selection

σ account.account-number = depositor.account-number (Account X Depositor)

و لكن نتيجة هذه العملية تعطي جميع الصفوف المحققة للشرط متضمنة الأعمدة الخمسة و لكننا نحتاج فقط الى عمود اسم الزبون :

عملية اسقاط projection

π customer-name (σ account.account-number = depositor.account-number (Account X Depositor))

ثانياً: الزبائن التي لديها قروض:

بنفس الطريقة التي حصلنا فيها على الزبائن التي لديها حسابات:

π customer-name (σ loan.loan-number = borrower.loan-number (Loan X Borrower))

و بما أن المطلوب هو الزبائن التي لديها قروض أو الزبائن التي لديها حسابات :

✂ عملية اجتماع

$$\begin{aligned} & [\pi \text{ customer-name } (\sigma \text{ loan.loan-number} \\ & = \text{borrower.loan-number} (\text{Loan X Borrower})) \cup \\ & \pi \text{ customer-name } (\sigma \text{ account.account-number} \\ & = \text{depositor.account-number} (\text{Account X Depositor}))] \end{aligned}$$

4. أسماء الزبائن التي لديها حسابات و قروض:

نفس الطلب السابق ولكن العملية تقاطع .

5. أسماء الزبائن التي لديها قرض في فرع المزة:

تذكر: الضرب الديكارتي دائما يولد أسطر غير مقبولة لذلك يجب وضع شرط قبلها (لفلترتها).

طريقة الحل الأولى :

✂ الزبائن التي لديها قرض:

$$\sigma \text{ loan.loan-number} = \text{borrower.loan-number} (\text{Loan X Borrower})$$

✂ نريد منها الأسطر التي تتضمن فرع المزة:

$$\begin{aligned} & \sigma \text{ branch-name} = \text{Mazzeh } (\sigma \text{ loan.loan-number} \\ & = \text{borrower.loan-number} (\text{Loan X Borrower})) \end{aligned}$$

حصلنا على جدول يحوي جميع الزبائن التي لديها قروض في فرع المزة تتضمن جميع الأعمدة من جدولي القروض و المقترضين و لكننا نحتاج فقط أسماء الزبائن :

✂ نجري عملية اسقاط على عمود customer-name :

$$\begin{aligned} & \pi \text{ customer-name } [\sigma \text{ branch-name} \\ & = \text{Mazzeh } (\sigma \text{ loan.loan-number} = \text{borrower.loan-number} (\text{Loan X Borrower}))] \end{aligned}$$

طريقة الحل الثانية :

بدلاً من إجراء عملية selection لجلب الزبائن التي لديها قرض و من ثم إجراء عملية selection أخرى على هذه النتيجة لجلب الأسطر التي الفرع فيها = المزة يمكن الدمج بين الشرطين في عملية selection واحدة :

$\sigma \text{ loan.loan-number} = \text{borrower.loan-number} \wedge \text{branch-name} = \text{MazzeH}$ (Loan X Borrower)

و من ثم نطبق عملية الاسقاط للحصول على أسماء الزبائن :

نجري عملية اسقاط على عمود customer-name :

$\pi \text{ customer-name } (\sigma \text{ loan.loan-number} = \text{borrower.loan-number} \wedge \text{branch-name} = \text{MazzeH} \text{ (Loan X Borrower)})$

طريقة الحل الثالثة :

نلاحظ وجود ال branch-name في ال loan و فعليا نحن بحاجة الأسطر التي فيها branch-name=MazzeH من جدول ال loan و ليس كل أسطره بالتالي بدلا من إجراء الضرب الديكارتي بين ال loan بجميع أسطره و ال Borrower نقوم بإجراء عملية selection على جدول ال loan لجلب الأسطر التي تتضمن فرع المزة فقط :

(نحتاج من جدول loan اسم الفرع فقط):

نقوم بإجراء عملية selection على جدول ال loan :

$\sigma \text{ branch-name} = \text{MazzeH}$ (Loan)

نجري الضرب الديكارتي بين ناتج هذه العملية و جدول المقرضين:

نجري الضرب الديكارتي:

$\sigma \text{ branch-name} = \text{MazzeH}$ (Loan) X Borrower)

و نختار من النتيجة الأسطر التي تحقق :
(رقم القرض في جدول القروض = رقم القرض في جدول المقرضين) .

نجري عملية اختيار selection

$\sigma \text{ loan.loan-number} = \text{borrower.loan-number} (\sigma \text{ branch-name} = \text{MazzeH} \text{ (Loan) X Borrower})$

✂ من ثم نجري عملية إسقاط للحصول على أسماء الزبائن:

π customer-name (σ loan.loan-number
=borrower.loan-number (σ branch-name=Mazzeh (Loan) X Borrower)))

6. أسماء الزبائن الذين لديهم قرض في فرع المزة و ليس لديهم حساب بأي فرع آخر :

نريد حدوديات موجودة في المجموعة الأولى و غير موجودة في المجموعة الثانية
✂ نجري عملية فرق

و في الطلب السابق حصلنا على الزبائن الذين لديهم قروض في فرع المزة :

✂ نقوم بإسناد النتيجة السابقة الى X لاستخدامها في هذا الطلب دون إعادة كتابتها كاملة:

توضيح: نقوم بإسناد نتيجة الزبائن الذين لديهم قروض في المزة إلى X ،
ثم نحذف منها الزبائن الموجودين في Depositor

$X \leftarrow (\pi$ customer-name (σ branch-name
=Mazzeh (σ loan.loan-number=borrower.loan-number (Loan X Borrower))))

تذكر: شرط الاجتماع و التقاطع و الفرق : نفس عدد الأعمدة + نفس النوع لذلك لا يمكن اجراء الفرق مباشرة

✂ نجري عملية الفرق :

$X - \text{Depositor}$ ☒

$X - \pi$ customer-name (Depositor) ☑

7. أكبر رصيد من الحسابات :

هذا النوع من الحسابات في الجبر العلاقتي يتطلب ضرب الجدول بنفسه .

فكرة الحل : بما أن الجبر العلاقتي لا يدعم دوال مثل MAX، (دالة موجودة في sql server ستتعرف عليها لاحقا)
فإننا نلجأ إلى أسلوب يُسمى المقارنة الذاتية. نقارن كل حساب مع كل حساب آخر، ونستبعد الحسابات التي لها أرصدة أصغر. الحساب الذي لا يوجد من هو أكبر منه، هو الحساب صاحب أكبر رصيد.

الفكرة النظرية للطلب & الحل :

الجبر العلاقتي لا يحتوي على دوال مثل MAX()، لذلك نحصل على الرصيد الأكبر من خلال استبعاد جميع الأرصدة التي يوجد ما هو أكبر منها. والقيمة الوحيدة التي تبقى هي أكبر رصيد موجود.

خطوات الحل بالتفصيل :

1. إعادة تسمية جدول Account مرتين :

 $\rho A1(Account), \rho A2(Account)$

هذا يسمح لنا بمقارنة رصيد كل صف في A1 مع رصيد آخر في A2.

اسناد النتيجة إلى T

2. تحديد كل الأرصدة التي يوجد ما هو أكبر منها :

 $T \leftarrow \sigma A1.balance < A2.balance (A1 \times A2)$

هذا الجدول يحتوي على جميع الصفوف من A1 التي يوجد ما هو أكبر منها في A2

3. إسقاط القيم (الأرصدة) الأصغر :

 $SmallerBalances \leftarrow \pi A1.balance(T)$

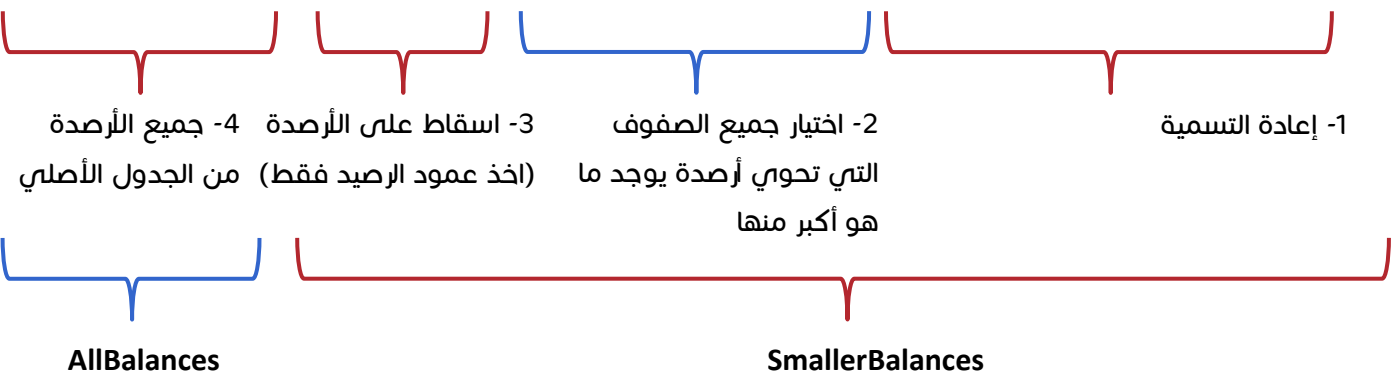
4. جميع الأرصدة (من الجدول الأصلي) :

 $AllBalances \leftarrow \pi balance (Account)$

5. طرح الأرصدة الأصغر من جميع الأرصدة → النتيجة هي الرصيد الأكبر فقط:

 $\pi balance(Account) - \pi A1.balance(\sigma A1.balance < A2.balance(A1 \times A2))$

أو بشكل مبسط بعد الإسناد:

 $AllBalances - SmallerBalances$ الحل في سطر : $\pi balance (Account) - \pi A1.balance (\sigma A1.balance < A2.balance (\rho A1 (Account) \times \rho A2 (Account)))$ 

مثال تطبيقي :

account-number	balance
101	5000
102	3000
103	7000
104	4000

تطبيق الخطوات على جدول مع قيم ك مثال
للتحقق من صحة الطريقة .

جدول Account الأساسي :

الخطوة الأولى : إعادة تسمية الجدول مرتين :

- A1 نسخة أولى من الجدول
- A2 نسخة ثانية من نفس الجدول

A1.account	A1.balance	A2.account	A2.balance
101	5000	101	5000
101	5000	102	3000
101	5000	103	7000
101	5000	104	4000
102	3000	101	5000
102	3000	102	3000
102	3000	103	7000
102	3000	104	4000
103	7000	101	5000
103	7000	102	3000
103	7000	103	7000
103	7000	104	4000
104	4000	101	5000
104	4000	102	3000
104	4000	103	7000
104	4000	104	4000

الخطوة الثانية تنفيذ الضرب الديكارتي الكامل :

$A1 \times A2$

سنربط كل صف من A1 مع كل صف من A2 ،
مما ينتج $16 = 4 \times 4$ صفًا .

الآن لدينا 16 صفًا تمثل كل زوج من الأرصدة الممكنة
بين A1 و A2 .

الخطوة الثالثة تطبيق شرط الاختيار $A1.balance < A2.balance$:

نُبقي فقط الصفوف التي يكون فيها رصيد A1 أقل من رصيد A2 :

A1.account	A1.balance	A2.account	A2.balance
101	5000	103	7000
102	3000	101	5000
102	3000	103	7000
102	3000	104	4000
104	4000	101	5000
104	4000	103	7000
101	5000	103	7000

هذه الصفوف تمثل كل الأرصدة التي يوجد ما هو أكبر منها.

الخطوة الرابعة إسقاط عمود A1.balance فقط :

balance
5000
3000
3000
3000
4000
4000
5000

نأخذ الأرصدة التي وُجد ما هو أكبر منها:

الخطوة الخامسة طرح الأرصدة الأصغر من كل الأرصدة الأصلية :

الأرصدة الأصلية:

balance
5000
3000
7000
4000

الأرصدة التي لها أكبر منها:

balance
5000
3000
4000

الفرق (أي أكبر رصيد فقط):

balance
7000

النتيجة النهائية ⌚ :

7000 هو أكبر رصيد موجود في الجدول

ملاحظات :

- ✓ الضرب الديكارتي هو توليد كل التراكيب الممكنة بين رصيد ورصيد.
- ✓ الاختيار يصفى الصفوف ويحدد من "الأصغر من غيره".
- ✓ الفرق يُستخدم للوصول إلى من "لا يوجد من هو أكبر منه".

Make your DB a cluster



8. الزبائن الذين لديهم حسابات في فرع المزة و فرع دويلعة :

أولاً الربط بين Depositor و Account للحصول على معلومات الفروع التي يملك الزبون حساباً فيها :

$\text{Depositor} \propto \text{Account}$

الآن نحتاج أن نجد الزبائن الذين ظهرت مرة على الأقل في المزة و مرة على الأقل في دويلعة :

ننشئ علاقتين:

$\text{InMezzeh} = \pi_{\text{customer-name}}(\sigma_{\text{branch-name} = \text{"المزة"}}(\text{Depositor} \propto \text{Account}))$

$\text{InDweila} = \pi_{\text{customer-name}}(\sigma_{\text{branch-name} = \text{"دويلعة"}}(\text{Depositor} \propto \text{Account}))$

ثم نأخذ التقاطع بينهما :

$\text{InMezzeh} \cap \text{InDweila}$

الناتج هو: الزبائن الذين لديهم حسابات في كلا الفرعين.

9. الزبائن الذين لديهم حسابات بكل أفرع دمشق :

هذا الطلب يستخدم عملية القسمة العلاقية (Relational Division)

وهي مثالية لمثل هذا النوع من الأسئلة :

🔑 فهم المطلوب لغوياً و منطقياً :

لاحظ هل السؤال يقول :

• "الزبائن الذين لديهم حسابات في بعض فروع دمشق" (لا)

• "الزبائن الذين لديهم حسابات في فرع معين" (لا)

بل يقول:

• "الزبائن الذين لديهم حسابات في جميع فروع دمشق"

نريد الزبائن الذين يملكون حسابات في جميع فروع دمشق، لا في فرع أو بعض الفروع فقط.

هذه عبارة شاملة (For All) وهذا النوع من الأسئلة لا يحله التقاطع ولا الاتحاد، بل نحتاج فيه

إلى عملية تحقق شاملة .

هل الزبون لديه حساب في كل فرع موجود في دمشق ؟

هذه الجملة تترجم في الجبر العلاقتي إلى استخدام:

الخطوة 1: ما الفروع التي تقع في "دمشق"؟

لأن السؤال يقيّدنا فقط بفروع دمشق.

من جدول Branch نحدد فروع دمشق:

$$\text{BranchesInDamascus} = \pi \text{ branch-name } (\sigma \text{ branch-city} = \text{"damascus"} (\text{Branch}))$$

هذه العلاقة تحتوي على أسماء جميع الفروع التي يجب أن يغطيها الزبون.

الخطوة 2: من لديه حساب في فرع معين؟

البيانات موزعة على Depositor و Account، لذا يجب ربطهم:

$$\text{Depositor} \infty \text{Account}$$

هذا يعطينا: (اسم الزبون، رقم الحساب، اسم الفرع، الرصيد...)

توضيح :

customer-name	account-number	branch-name	balance

شكل الجدول الناتج عن الانضمام سيكون كالتالي:

شرح الأعمدة:

customer-name ← من جدول Depositor

account-number ← العمود المشترك (موجود في كلا الجدولين، يظهر مرة واحدة فقط في الناتج)

branch-name ← من جدول Account

balance ← من جدول Account

ملاحظة : لا يتم تكرار العمود المشترك (account-number/number) مرتين.

الآن نُسقط فقط اسم الزبون واسم الفرع :

$$\pi \text{ customer-name, branch-name } (\text{Depositor} \infty \text{Account})$$

هذه العلاقة تخبرنا : الزبون X يملك حساباً في الفرع Y

الخطوة 3: نريد فقط الزبائن الذين لديهم فرع في جميع النتائج

التي ظهرت في خطوة BranchesInDamascus :

وهذا يتحقق بعملية:

$$\pi \text{ customer-name } (\text{Depositor} \infty \text{Account}) \div \pi \text{ branch-name } (\sigma \text{ branch-city} = \text{damascus} (\text{Branch}))$$

ما الذي يعنيه هذا فعلياً !

يسأل :من هم الزبائن الذين تظهر أسماؤهم مرتبطة بكل فرع من فروع دمشق؟
القسمه العلاقية تتحقق من أن كل عنصر (زبون) في العلاقة الأولى يغطي كل القيم الموجودة في العلاقة الثانية.



فائدة أربيسيزية !

عندما يُطلب منك "من لديه عنصر يرتبط بجميع عناصر مجموعة أخرى بالكامل، وليس ببعضها فقط. ؟"
فإنك تستخدم عملية القسمه العلاقية

ملخص نهائي :

الهدف	الرمز	العملية
استخراج صفوف تحقق شرطاً معيناً	σ	الاختيار
تحديد الأعمدة المطلوبة فقط	π	الإسقاط
دمج صفوف جدولين دون تكرار	$\cup$	الاتحاد
استخراج الصفوف المشتركة فقط	$\cap$	التقاطع
استخراج الصفوف الموجودة في جدول وغير موجودة في جدول آخر	$-$	الفرق
كل التركيبات الممكنة بين جدولين	$\times$	الضرب الديكارتي
تعديل اسم الجدول أو أعمدته	ρ	إعادة التسمية
ربط جدولين عبر شرط مشترك	∞	الانضمام
الإجابة على أسئلة "لكل"...	$\div$	القسمه
تخزين نتيجة مؤقتة باسم	$\leftarrow$	الإسناد



انتهت المحاضرة الرابعة 🚀